TECHNICAL ARCHITECTURE SCOPE

AfryStride Sportswear — Stack, Database & File Structure

Companion document to the Design & Development Scope — covers what runs the site under the hood.

1. Purpose of This Document

This document defines the technology stack, database design, and project file structure for the AfryStride website — both Phase 1 (business/advertising site) and Phase 2 (full e-commerce). It's the technical companion to the Design & Development Scope: that document defines how the site looks and flows, this one defines how it's actually built and organised, so development can start from an agreed structure instead of being figured out as we go.

2. Technology Stack

Layer	Technology	Why
Frontend	React (Vite) + Tailwind CSS	Fast build tooling, matches the component-driven design system from the Design Scope, easy to hand off/maintain
Backend / API	FastAPI (Python)	Fast to build, automatic API docs, async support for handling image uploads and future payment webhooks
Catalog Database	MongoDB	Flexible schema — a jersey and a pair of shoes have different attributes (size, team, colour); documents handle that better than rigid columns
Transactional Database (Phase 2)	PostgreSQL	Orders, payments, and stock counts need strict consistency (ACID) — money and inventory should never be 'eventually correct'
Authentication	JWT (admin), OAuth2 password flow	Secures the admin panel; customer accounts (Phase 2) reuse the same pattern
Image Storage	Cloudinary or Render disk + CDN	Product photos need to load fast on mobile — offload to a CDN rather than serving raw from the API
Backend Hosting	Render	Consistent with your existing deployment pattern; free tier viable for Phase 1 traffic
Frontend Hosting	Hostinger	Matches your current hosting setup
Payments (Phase 2)	M-Pesa Daraja API (primary), Stripe (secondary)	Daraja covers the primary local payment method; Stripe covers card/diaspora buyers

3. Database Design

A hybrid approach: MongoDB for anything product-catalog related (flexible, changes often), PostgreSQL for anything transactional (fixed structure, must always add up correctly). This mirrors the pattern already used on Malaki AI, so the workflow stays familiar.

3.1 MongoDB — Product Catalog (Phase 1 onward)

Collection	Key Fields	Notes
products	name, slug, category, price, sizes[], colours[], images[], stock_status, tags[], created_at	tags[] powers badges like 'New' / 'Best Seller' / 'School Uniform'
categories	name, slug, parent_category, display_order	Supports the flat nav + dropdown structure from the Design Scope
admins	email, hashed_password, role, last_login	Single collection is enough at this scale — no need for a separate roles table yet
contact_messages	name, phone, message, submitted_at, status	From the Contact page form

3.2 PostgreSQL — Transactions (Phase 2)

Table	Key Fields	Notes
customers	id, name, phone, email, password_hash, created_at	Customer accounts, separate from admins table
orders	id, customer_id, status, total_amount, delivery_address, created_at	status: pending / paid / shipped / delivered / cancelled
order_items	id, order_id, product_id (ref Mongo _id), quantity, size, unit_price	Bridges to MongoDB product_id — product details stay in Mongo, order facts stay in Postgres
payments	id, order_id, method (mpesa/stripe), transaction_ref, amount, status	Stores Daraja/Stripe transaction references for reconciliation

3.3 Why split the databases instead of using one?

- Product attributes change shape often (a shoe needs sizes 38–46, a jersey needs S–XXL, an accessory needs neither) — MongoDB handles that without constant schema migrations.
- Money and stock counts must never be ambiguous — PostgreSQL's strict schema and transactions prevent double-charging or lost orders.
- order_items.product_id simply references the Mongo product's _id as a string — no need for a full relational join, just enough to look up product name/image when displaying an order.

4. Project File Structure

Both the backend and frontend follow a nested, feature-oriented structure — each concern (routing, business logic, data models, UI components) lives in its own folder rather than everything sitting in one flat directory. This keeps the codebase navigable as Phase 2 features (cart, payments, customer accounts) get added, and makes it easy to tell at a glance whether something belongs to Phase 1 or Phase 2.

4.1 Backend — FastAPI

```
afrystride-backend/
├── app/
│   ├── main.py                    # FastAPI app entrypoint
│   ├── core/
│   │   ├── config.py              # env vars, settings (pydantic BaseSettings)
│   │   ├── security.py            # JWT auth, password hashing (admin login)
│   │   └── database.py            # Mongo + Postgres connection setup
│   │
│   ├── api/
│   │   ├── v1/
│   │   │   ├── router.py          # combines all v1 routers
│   │   │   ├── products.py        # GET/POST/PUT/DELETE /products
│   │   │   ├── categories.py      # /categories
│   │   │   ├── admin.py           # admin auth + dashboard endpoints
│   │   │   ├── contact.py         # contact form submissions
│   │   │   ├── orders.py          # Phase 2: order lifecycle
│   │   │   └── payments.py        # Phase 2: M-Pesa Daraja + Stripe webhooks
│   │
│   ├── models/
│   │   ├── mongo/
│   │   │   ├── product.py         # Product document schema
│   │   │   └── category.py
│   │   └── sql/
│   │       ├── order.py           # Phase 2: SQLAlchemy models
│   │       ├── customer.py
│   │       └── payment.py
│   │
│   ├── schemas/                   # Pydantic request/response schemas
│   │   ├── product_schema.py
│   │   ├── order_schema.py
│   │   └── admin_schema.py
│   │
│   ├── services/                  # business logic, kept out of routers
│   │   ├── product_service.py
│   │   ├── order_service.py       # Phase 2
│   │   ├── mpesa_service.py       # Phase 2: Daraja STK push, callbacks
│   │   └── stripe_service.py      # Phase 2
│   │
│   └── utils/
│       ├── image_upload.py        # product image handling
│       └── validators.py
│
├── tests/
│   ├── test_products.py
│   └── test_orders.py
│
├── .env                           # local secrets (never committed)
├── .env.example
├── requirements.txt
└── render.yaml                    # Render deployment config
```

- api/v1/ — one router file per resource; router.py combines them so main.py stays clean
- models/mongo/ vs models/sql/ — physically separates the two databases so it's obvious at a glance which storage a piece of data lives in

- services/ — business logic (e.g. "what happens when an order is placed") lives here, not in the route handlers, so routes stay thin and logic is testable/reusable
- Phase 2-only files (orders.py, payments.py, mpesa_service.py, sql/ models) are already placed in the structure now, just left unbuilt — so adding Phase 2 later means filling in files, not restructuring the project

4.2 Frontend — React

```
afrystride-frontend/
├── public/
│   ├── logo.svg
│   └── favicon.ico
│
├── src/
│   ├── main.jsx
│   ├── App.jsx
│   │
│   ├── assets/
│   │   ├── images/
│   │   └── icons/
│   │
│   ├── components/
│   │   ├── layout/
│   │   │   ├── Header.jsx
│   │   │   ├── Footer.jsx
│   │   │   └── StickyOrderBar.jsx
│   │   ├── product/
│   │   │   ├── ProductCard.jsx
│   │   │   ├── ProductGrid.jsx
│   │   │   └── ProductGallery.jsx
│   │   ├── ui/                    # shared building blocks
│   │   │   ├── Button.jsx
│   │   │   ├── Badge.jsx
│   │   │   └── FilterPanel.jsx
│   │   └── cart/                  # Phase 2
│   │       ├── CartDrawer.jsx
│   │       └── CheckoutForm.jsx
│   │
│   ├── pages/
│   │   ├── Home.jsx
│   │   ├── Shop.jsx               # category listing
│   │   ├── ProductDetail.jsx
│   │   ├── About.jsx
│   │   ├── Contact.jsx
│   │   └── admin/
│   │       ├── AdminLogin.jsx
│   │       ├── AdminDashboard.jsx
│   │       └── AdminProducts.jsx
│   │
│   ├── hooks/
│   │   ├── useProducts.js
│   │   └── useCart.js             # Phase 2
│   │
│   ├── services/                  # API calls, one file per resource
│   │   ├── api.js                 # axios instance, base URL, interceptors
│   │   ├── productService.js
│   │   └── orderService.js        # Phase 2
│   │
│   ├── context/
```

```
│   │   ├── AuthContext.jsx        # admin auth state
│   │   └── CartContext.jsx        # Phase 2
│   │
│   ├── styles/
│   │   └── tailwind.css
│   │
│   └── utils/
│       └── formatCurrency.js
│
├── .env
├── .env.example
├── tailwind.config.js
├── vite.config.js
└── package.json
```

- components/ split into layout/, product/, ui/, cart/ — matches the reusable component approach from the Design Scope (Button, ProductCard, etc. built once, used everywhere)
- pages/ holds one file per route, kept thin — pages assemble components, they don't contain business logic
- services/ — all API calls centralised here, so if an endpoint changes, it's a one-file fix, not a hunt through every page
- context/CartContext.jsx is scaffolded now but unused until Phase 2 — no rework needed to wire the cart in later
- admin/ pages are separated from public pages so admin-only code and public storefront code stay clearly divided

5. Environment & Configuration

- .env files hold all secrets (database URLs, JWT secret, M-Pesa/Stripe keys) — never committed to version control, .env.example committed instead as a template
- Separate .env values for local development vs Render production, switched via Render's environment variable dashboard
- CORS locked to the Hostinger frontend domain in production, open to localhost during development

6. Deployment Overview

Component	Host	Notes
Frontend (React build)	Hostinger	Static build output deployed via Hostinger's file manager or Git deploy
Backend (FastAPI)	Render	Web service, auto-deploys from GitHub on push to main
MongoDB	MongoDB Atlas (free tier to start)	Managed, no server maintenance needed
PostgreSQL (Phase 2)	Render Postgres or Supabase	Managed relational DB, added when Phase 2 begins
Images	Cloudinary	Free tier sufficient for Phase 1 product volume

Repos stay separate (afrystride-backend, afrystride-frontend) rather than a monorepo — matches your existing project pattern and keeps Render/Hostinger deploys independent of each other.